Università di Pisa

Dipartimento di Informatica

Shakespeare Interpreter in Go

Validation Document

How the Generated Output Was Tested, Verified,
and Proven Specification-Conformant

Author
Lorenzo Bandini

Course
Advanced Programming
A.Y. 2025/2026

Professors
Andrea Corradini
Antonio Cisternino

July 22, 2026

Contents

This document discusses how the AI-generated code for the SPL interpreter was tested, verified, and refined. It follows the first-person reflective tone established by the companion Prompts Document and mirrors the testing methodology recommended in the course materials.

The complete project lives at https://github.com/lorenzobandini/shakespeare-interpreter-go. Every file, function, error code, and fixture named in this document is real and can be inspected at the linked repository.

1 Testing and Validation Methodology

The core development process followed a strict Generate-Verify-Accept loop, adapted from the iterative refinement workflow described in the course's example validation report:

1. **Generate:** Issue a specific, small-scope request to the LLM agent within the plan-mode scaffold.
2. **Verify:** Run the single quality gate `task check` (fmt → lint → vuln → test with `-race`) and inspect the output.
3. **Accept:**
   - **Accept:** If the gate is green and the behaviour matches expectations, record decisions in `PROGRESS.md` and commit.
   - **Revert-and-Retry:** If the gate fails or behaviour is incorrect, revert the changes (the failed fragment stays out of the buffer) and refine the prompt.

This methodology proved more effective than traditional debugging: reverting and re-prompting was faster than diagnosing and fixing generated code, and it kept the commit history clean and bisectable.

2 Testing Architecture Overview

2.1 Test Runner

The single quality gate is `task check`, defined in `Taskfile.yaml`, chaining formatting, linting, vulnerability scanning, and the full test suite in order. The exact tools, flags, and enabled checks behind each stage are listed once, as a report card, in Section 8; this section only establishes that the gate exists and is enforced identically by `lefthook.yml` (pre-commit, serial) and `.github/workflows/ci.yml`, so no change can reach `main` without passing the same checks a local commit would.

2.2 Golden-Snapshot Discipline

Two golden-snapshot patterns are used across the project, and every later reference to "golden tests" in this document means one of these two:

- `internal/runtime/golden_test.go` uses an explicit `-update` flag: `go test -update ./internal/runtime/...` overwrites the golden file with the current output.
- Lexer and parser golden tests (`TestGoldenSnapshots`) use auto-create-then-fail-on-first-run: the very first time a new fixture is added, no golden file exists yet, so the test creates one from the current output but still reports FAIL for that run, forcing a manual check that the newly created file is actually correct before later runs start trusting it silently.

Both patterns share the same intent: a golden file changing is always a decision the developer makes explicitly, never a side effect.

2.3 Logger Initialisation

Per-package `TestMain` calls `logger.Init(level)`. Every internal package, `lexer`, `parser`, `semantic`, `runtime`, uses `LevelDebug`, so state transitions (token emission, symbol-table inserts/lookups, stage mutations) are fully visible when diagnosing a failing test. Only `cmd/spl`'s CLI-level tests use `LevelInfo`, since those tests exercise the whole pipeline through the public command surface and do not need internal engine-level tracing.

3 Error Taxonomy Glossary

The test suite exercises every error code defined in `docs/spl/errors.md` (L001-L002, S001-S019, M001-M008, R000-R005) where letters stand for: L for Lexer, S for Syntax (the parser), M for Meaning (semantics), R for Runtime. The table below is the single authoritative index mapping each code to its triggering condition and the test that validates it.

Code	Phase	Condition	Key Test(s)
L001	Lexer	Control character in source	`TestScanTokensControlCharL001`
L002	Lexer	Unterminated `[` at EOF	`TestScanTokensL002UnterminatedBracket`
S001	Parser	Empty source	`TestParseErrorS001Empty`
S002	Parser	No character declaration before Act I	`TestParseErrors`
S003	Parser	Non-Shakespearean name (warning)	`TestWarningIsNotError`
S004-S018	Parser	Various structural violations	`TestParseErrors` table-driven, `TestGoldenSnapshots`
S019	Parser	Unconditional goto	—
M001	Semantic	Undeclared character	`TestM001UndeclaredEnter`, `TestM001UndeclaredSpeaker`
M002	Semantic	>2 characters in a single Enter	`TestSemanticErrorFormat`
M003	Semantic	>2 characters on stage	`TestM003StageOverflow`
M004	Semantic	Speaker not on stage	`TestM004SpeakerNotOnStage`, `TestM004EmptyStageCrossAct`
M005	Semantic	Exit not on stage	`TestM005ExitNotOnStage`
M006	Semantic	Undefined Act/Scene	`TestM006UndefinedScene`, `TestM006UndefinedAct`
M007	Semantic	Re-enter already-on-stage character	`TestM007SelfEnter`
M008	Semantic	Defensive overlap with S007	`TestAnalyzeEmptyScenesDefensive`
R000	Runtime	Internal guard (recover wrap)	`TestEvalCharRefDefensive`, `TestEvalPronounDefensive`
R001	Runtime	Division/modulo by zero	`TestEvalBinary`, `TestExecuteDivZero`
R002	Runtime	Non-numeric input to Listen	`TestExecListen`
R003	Runtime	EOF during I/O read	`TestExecOpenMind`, `TestExecListen`
R004	Runtime	Factorial > 20	`TestEvalUnary`
R005	Runtime	Max steps exceeded	`-max-steps` flag acceptance test

4 Per-Phase Testing Narrative

This section explains *why* certain tests exist rather than just naming them: the design decisions, the tricky edge cases, and the bugs they were written to catch.

4.1 Lexer (`internal/lexer/`)

The lexer was designed under the "dumb lexer" philosophy: it only recognises structural punctuation (`.` `,` `:` `!` `?` `[` `]`) and foldable separators (apostrophe, dash), leaving all word classification to the parser. The interesting edge case is the L001 boundary: standalone `A` and `S` (which could be articles or character-name fragments) and the `@` sign are all deliberately accepted as ordinary WORD content rather than triggering an error, because L001 is scoped to actual control characters, not to printable punctuation that might look unusual.

Case handling follows the same philosophy: the lexer preserves the original mixed-case text in every token (for debug output) while classification elsewhere in the pipeline is case-insensitive. Concretely: whether the source says `ROMEO`, `Romeo`, or `romeo`, the token keeps the exact original spelling (so error messages show precisely what the user wrote), while later stages compare names case-insensitively, meaning a lowercase `romeo` in dialogue still correctly matches a character declared as `Romeo`.

4.2 Parser (`internal/parser/`)

The parser test suite is the most extensive in the project, structured around two convenience functions, `parseExprStr` and `parseStmtStr`, which enable pure unit testing of expressions and statements without building complete SPL programs.

The single most important regression in this phase is the `enterSeen` bug. The parser tracks a flag recording whether any `[Enter]` directive has appeared, but it was originally re-created inside the function that parses a *single* act, so it reset every time a new act began. This caused a false S013 error whenever characters legitimately carried over from a previous act without an explicit closing `Exeunt`. The fix moved the flag's declaration up to `parseActs()`, the outer function that iterates over *all* acts, so a single instance now persists across act boundaries instead of resetting at the start of every act. `TestParseCrossActPersistence` together with the `cross-act-persistence.spl` fixture guards against this regressing.

Expression and statement coverage is otherwise systematic: every arithmetic operator, every dialogue form, and both assignment forms (with and without comma) are tested, including the 6-adjective insult form that evaluates to -64 under the $value = polarity \times 2^{adjCount}$ formula. Nouns are classified as positive or negative (polarity $+1$ or -1); each adjective placed before the noun doubles the magnitude. So `a flower` (positive, zero adjectives) $= +1 \times 2^0 = 1$; `a pretty flower` (one adjective) $= +1 \times 2^1 = 2$; `a vile coward` (negative, one adjective) $= -1 \times 2^1 = -2$.

The one case worth calling out specifically is `TestWarningIsNotError` where a non-Shakespearean character name like `Coriolanus` produces an S003 *warning*, not a hard error, confirming that the specification does not strictly enforce Shakespearean names.

4.3 Semantic Analysis (`internal/semantic/`)

The semantic analyser exercises every M-code from M001 through M008, but three design decisions are worth understanding rather than just listing:

Decision D1 (off-stage value reads allowed): a character's *value* can be read in an expression while that character is off stage, even though only the *speaker* of a dialogue must actually be present.

Decision D2 (stage persists across acts): the stage does not clear at act boundaries. `TestAnalyzeWalksMultipleActsWithPersistence` confirms the analyser enforces this over `primes-persistence.spl`.

Decision D3 (collect-all error mode) means the analyser never fails fast; like a linter, it keeps checking after the first error and returns every problem it can find from a single run, rather than stopping the caller at the first one.

4.4 Runtime (`internal/runtime/`)

The runtime test suite is split across six files: `eval_test.go` (expression evaluation), `exec_test.go` (statement execution), `execute_test.go` (full-program execution), `errors_test.go` (error formatting), `golden_test.go` (end-to-end golden snapshots) and `main_test.go` (per-package test).

Behaviour tested	File	Detail
Enter/Exit/Exeunt	`exec_test.go`	Re-enter after backward goto silently no-ops; a failed Exit is discarded, not surfaced
Speak / Open heart / Open mind / Listen (I/O)	`exec_test.go`	1-byte output, decimal + newline output, EOF → R003, non-numeric input → R002
Self-talk I/O	`exec_test.go`	Listener resolves to the speaker when alone on stage
Remember/Recall	`exec_test.go`	Listener-versus-speaker asymmetry; `Remember yourself` (listener pronoun in an expression)
Comparatives, If/Goto	`exec_test.go`	All 6 relations table-driven; scene- and act-target jumps
Flatten + label maps	`execute_test.go`	Scene → program-counter mapping
Full linear execution + div-by-zero	`execute_test.go`	End-to-end pipeline from source; R001 with Act/Scene in the message
Rejects unvalidated AST	`execute_test.go`	Security gate, see below
7 canonical E2E fixtures	`golden_test.go`	hello, branch, stack, io-ascii, truth-machine (×2), divzero

Two design points deserve attention beyond the taxonomy mapping.

First, the listener-versus-speaker asymmetry in stack operations: `Remember` pushes onto the *listener's* stack, while `Recall` pops from the listener's stack but assigns into the *speaker*.

`TestRememberRecallRoundTrip` exercises both directions together, and an empty-stack `Recall` is defined to produce 0 rather than erroring.

Second, `TestExecuteRejectsUnvalidatedAST` is a security-relevant test, not just a functional one: it proves the execution engine refuses to run a `semantic.Result` carrying a synthetic, invalid error code, meaning the `Execute` entry point trusts only analyses that have already passed semantic validation, and cannot be tricked into running an AST that skipped that gate.

5 End-to-End Integration

5.1 Golden Test Harness

The canonical E2E harness lives in `internal/runtime/golden_test.go`. `TestGoldens` is table-driven with seven cases, each reading a `.spl` file from `testdata/runtime/` and running the full production pipeline: `lexer.ScanTokens` → `parser.Parse` → `semantic.New(filename, prog).Analyze()` → `Execute(prog, res, in, out, filename)`. The output is compared byte-for-byte against the expected golden output (per the discipline established in Section 2.2).

Success cases:

- `hello`: stdout is a single non-printable byte, `0x02` (ASCII STX)
- `branch`: stdout `1\n`.
- `stack`: stdout `1\n0\n0\n`.
- `io-ascii`: stdin `X`.
- `truth-machine`: stdin `0\n` terminates normally.

Error cases:

- `truth-machine-1`: stdin `1\n` triggers R003 (EOF after N repetitions). This documents the infinite-loop-on-truthy behaviour that is inherent to the SPL truth-machine specification. The test uses `findSource`, a fallback that splits on the `-` character, to derive the source filename from the test name.
- `divzero`: R001 division by zero, with the error message naming "Act I, Scene I."

5.2 CLI-Level E2E

The CLI is tested through `cmd/spl/main_test.go`. `TestRunCommand` runs `spl run self-talk.spl` through the full Cobra pipeline. `TestRepl_StdinReplayAcrossSubmits` tests the REPL pipeline, asserting that `42` reaches stdout after a recorded stdin replay.

6 REPL Regression Episode

The REPL resolution is the canonical regression-prevention episode in this project. After the REPL was completed in Phase 5, real usage surfaced spurious errors: S002, S006, S007, S009, L002, and M004 appeared on multi-line pastes and incremental entry.

6.1 Root-Cause Isolation

The first diagnostic step was critical: failures were confined to `cmd/spl/main.go` REPL state management. The shared pipeline (used by `spl run`) ran flawlessly. The `AGENTS.md` design-protection clause was respected throughout, `runPipeline` and `internal/*` packages were never

touched. This confinement meant the fix could be aggressive within the REPL component without destabilising the rest of the system.

6.2 Regression Fixtures

Five failure logs from real REPL sessions were collected and converted into regression fixtures in `testdata/repl/`. Each fixture is a text driver consumed by `TestRepl_UserLogRegressions` with subtests:

- `title-only-no-s002`: Title alone does not fire spurious S002.
- `char-decl-and-body`: Character declaration followed by body parses correctly.
- `stack-test-body-only`: Stack operations without surrounding ceremony work.
- `act-without-scene-reports-s007`: Genuine S007 on malformed act-without-scene.

6.3 Phase-Tracking State Machine

The REPL was rewritten around a four-phase state machine called `replPhase`, a small fixed set of named states tracking which part of an SPL program the REPL believes the user is currently in: still writing the title, still declaring characters, inside the play body, or finished).

Unit tests cover the classifier: `TestClassifyLine`, `TestDerivePhase`, `TestIsOpenExpression`, `TestTryQuickParse_ErrorLines`. Only three signals make the REPL wait for more input rather than run what it has:

1. an unterminated bracket at the end of the block (e.g. `[Enter Romeo` with no closing `]`);
2. a block that is trivially short (too little text to plausibly be a complete statement);
3. an "open expression" heuristic. The last sentence does not yet end in `.`/`!`/`?`, suggesting the user is still mid-sentence.

Everything else is submitted immediately.

6.4 Skeleton-Injection Ordering

The skeleton (minimal Title + Act I + Scene I) is injected only at the first `phaseBody` submit, spliced after the last character-declaration byte. `TestRepl_SkeletonInjectionOrder` validates this ordering. `TestRepl_NoSkeletonWhenUserProvidesActs` proves the skeleton is skipped entirely if the user provides their own Act header.
`TestRepl_MissingCharDeclBeforeActI_ReportsS002` proves that the REPL produces an honest S002 (no synthetic character).

6.5 Auto-Declaration

`TestRepl_AutoDeclaration` and `TestRepl_AutoDeclarationMidProgram` prove that a new character referenced in a later submit is auto-declared without producing M001 or S002. The `extractChars` function scans the user's submitted text for referenced character names, returns them in first-seen order, and splices declarations immediately before the skeleton's Act I. Case-insensitive dedup prevents duplicate declarations, and only names the user's text already references are declared.

7 Audit Findings

The final comprehensive audit (Phase 8) examined the full repository across four axes: specification compliance and edge cases, state management and concurrency safety, Go idioms and code quality, error handling robustness. Fourteen findings were identified and resolved. This table is the single, authoritative record of every audit-driven fix; none of these items are repeated elsewhere in this document.

Step	Severity	Finding	Resolution
1	Critical	`SemanticError.Filename` never populated	`addErr` helper stamps filename at all ∼8–12 call sites
2	Critical	`Analyze()` reuse footgun: calling twice silently appends errors to non-reset slice	Single-use contract documented, `prog` param dropped from `Analyze()`
3	Medium	SIGINT not handled; Ctrl+C causes unclean shutdown	`signal.NotifyContext` replaces `os.Exit(0)`
4	Medium	`spl run` stdin hardcoded to `os.Stdin`; tests cannot inject input	Changed to `cmd.InOrStdin()`
5	Critical	No infinite-loop guard; truth-machine fed `1` runs forever	`maxSteps` fuel counter + R005 error + `-max-steps` CLI flag
6	Low	Factorial overflows on 32-bit architectures	`int64` internal arithmetic for all runtime values
7	Medium	Pronoun resolution incomplete; `you`/`thou`/`I` in expression position not resolved	Expanded pronoun lookup
8	Critical	Malformed AST causes panic during evaluation	`recover()` wrapper + R000 error
9	Low	Unconditional goto (`Let us proceed to scene X`) reports misleading S016	Dedicated S019 error code
10	Low	Float sqrt via `math.Sqrt` loses precision for values $> 2^{53}$	Newton iteration for exact integer sqrt
11	Low	Named Exeunt (`[Exeunt A and B]`) mutates stage partially on validation failure	Two-pass: validate all names first, then mutate

8 Validation Metrics

This is the project's report card: the concrete, reproducible numbers behind every claim made in the preceding sections.

8.1 Coverage

```
go test -coverprofile=coverage.out ./...
go tool cover "-func=coverage.out" # per-function summary
go tool cover "-html=coverage.out" # interactive browser report
```

Overall statement coverage is 81.9%. Per package:

Package	Coverage
`cmd/spl`	85.0%
`internal/lexer`	92.5%
`internal/parser`	72.7%
`internal/runtime`	86.3%
`internal/semantic`	97.5%

The parser pulls the average down, and it does so for a specific, low-risk reason: it carries most of the project's boilerplate, dozens of `Pos()`/`String()` AST-node methods used only for debug printing, error-sentinel constructors exercised only indirectly through the code that calls them, and a couple of dead-branch service methods (`parseSimileInExpr`, `intToRoman`, covered indirectly via `parseRoman`). Similar zero-coverage boilerplate exists in `logger.Init()` (called only from `main()`) and `cmd/spl/main.go`'s entry point itself, neither of which is directly testable.

8.2 Test Count

The suite comprises 154 top-level test functions across the five packages above, expanding to 333 individual test cases once table-driven subtests are counted. All 333 pass at submission time.

8.3 Race Safety

Every test runs with the `-race` flag (part of `task test`, Section 2.1). This guarantees concurrency safety for the single-goroutine trampoline and map-based environment. Zero race conditions have been reported.

8.4 Vulnerability Scan

```
task vuln # govulncheck ./... -- zero known advisories
```

8.5 Lint Cleanliness

`task lint` runs `golangci-lint v2` with the enabled set `errcheck`, `govet`, `ineffassign`, `staticcheck`, `unused`, `bodyclose` (timeout 3m). Clean at submission.

8.6 Reproducibility Commands

```
git clone https://github.com/lorenzobandini/shakespeare-interpreter-go
cd shakespeare-interpreter-go && task check

docker build --target=check -t spl:check .
```

CI green badge on `main` from `.github/workflows/ci.yml`.